

Rozgar ? Complete System Overview & Viva Guide

Semester Project | Mobile Application Development
Student App: Rozgar (Job Portal) | Version: 1.0.0

PART A ? What This System Does (Complete Description)

1. System Name & Purpose

Rozgar (Urdu/Hindi: "Livelihood/Employment") is a multi-role job portal mobile application built with Flutter. It connects three types of users:

Role | Who | What They Do

|---|---|---|

Job Seeker (User) | People looking for jobs | Browse jobs, apply, track status, chat with recruiters, manage profile

Recruiter (Company) | Hiring companies | Post jobs, review applicants, update application status, chat with candidates

Administrator (Admin) | System manager | View analytics, manage users/jobs, verify system diagnostics

2. Problem Statement

Traditional job searching is fragmented ? candidates cannot track applications, recruiters lack a unified posting platform, and sensitive data (phone, salary) is stored insecurely. Rozgar solves this with:

- Role-based access control
- Real-time application tracking
- Encrypted sensitive data storage
- In-app messaging between candidates and recruiters
- Hybrid job feed (company listings + external API jobs)

3. Technology Stack

Layer | Technology

|---|---|

Frontend | Flutter 3.11+ (Dart)

State Management | Flutter Riverpod

Navigation | GoRouter

Backend/Auth | Firebase Authentication

Database | Cloud Firestore (NoSQL)

Push Notifications | Firebase Cloud Messaging (FCM)

Performance Monitoring | Firebase Performance

Media Storage | Cloudinary CDN

External API | JSONPlaceholder (REST via Dio)

Encryption | AES-256 CTR (encrypt package)

Secure Storage | Flutter Secure Storage

Background Tasks | WorkManager

Threading | Dart Isolates

Ads | Google Mobile Ads (AdMob)

Permissions | permission_handler

4. System Architecture Diagram

??

? FLUTTER APP (Rozgar) ?

??

? Job Seeker ? Company ? Admin ? Shared Core ?

? (User UI) ? (Recruiter) ? Dashboard ? Services ?

??

? Riverpod State Layer ?

??
? Auth Repo ? Job Repo ? App Repo ? Chat ? Cloudinary ? Ads ?
??
? Firebase ? Firestore ? FCM ?Cloudinary?JSONPlace- ?
? Auth ? DB ? ? CDN ? holder API ?
??

5. Database Schema (Firestore)

users/{userId}

??? email, displayName, role, photoUrl
??? phone (ENCRYPTED), salary (ENCRYPTED)
??? skills, education, experience, headline

jobs/{jobId}

??? title, description, location, company, companyId
??? salary, tags, bannerUrl, isPremium, postedAt
??? comments/{commentId}
??? authorId, text, createdAt

applications/{applicationId}

??? jobId, jobTitle, companyName, companyId
??? applicantId, applicantName, experience, resumeUrl
??? status, appliedAt
??? status/{statusId}
??? status, label, timestamp

chats/{chatId}

??? participants[], title, lastMessage, lastMessageAt
??? messages/{messageId}
??? senderId, text, createdAt, status

6. Complete Feature List

Job Seeker Features

- Register/Login with email and password
- Browse hybrid job feed (Firestore premium + REST API jobs)
- View job details with banner image
- Like and comment on premium jobs
- Apply to jobs with experience and resume upload
- Real-time application status tracker (Applied ? Review ? Interview ? Offered)
- Chat with recruiters
- Edit profile (headline, skills, education, experience, encrypted salary)
- Upload profile photo and CV via Cloudinary
- Receive notifications on application submit

Recruiter (Company) Features

- Register as company role
- Post jobs via 3-step wizard (details ? compensation ? banner upload)
- View all posted jobs
- Review applicants for their company's jobs
- Update application status (approve, reject, schedule interview)
- Chat with candidates
- Edit company profile

Admin Features

- View registration analytics chart
- Browse all users and jobs
- System diagnostics (rubric verification + live debug logs)
- Admin profile management

System-Level Features

- AES-256 encryption for sensitive fields
- Firebase Performance profiling
- Structured logging with admin viewer
- WorkManager background tasks
- Dart Isolate threading
- Runtime permissions with rationale dialogs
- Google Mobile Ads (banner + skippable overlay)
- Dark/Light theme toggle
- Glassmorphic UI design

7. Main User Flows

Flow 1: Registration & Login

Splash ? Login/Register ? Firebase Auth ? Firestore Profile ? Role Home

- Role selected at registration: Job Seeker / Recruiter / Admin
- Router auto-redirects based on role

Flow 2: Job Discovery & Apply

Jobs Tab ? Job Detail ? Apply Sheet ? Upload Resume ? Submit
? Firestore applications + status ? Local Notification ? Track Status

Flow 3: Company Job Posting

Post Job Tab ? Step 1 (title, description) ? Step 2 (location, salary, tags)
? Step 3 (banner upload to Cloudinary) ? Publish to Firestore

Flow 4: Messaging

Job Detail ? Message Recruiter ? ensureChat() ? Chat Screen ? Send/Receive

Flow 5: Application Management (Company)

Applicants Tab ? View Applications ? Update Status ? User sees live update

PART B ? Complete Code Structure

Project Root

```
job_application/  
??? android/          # Android native config + permissions  
??? ios/              # iOS config + Info.plist permissions  
??? assets/icon/      # App icon  
??? docs/             # Documentation (this file)  
??? firestore.rules   # Firestore security rules  
??? firestore.indexes.json # Firestore composite indexes  
??? firebase.json     # Firebase project config  
??? pubspec.yaml      # Dependencies  
??? lib/              # All Dart source code
```

lib/ ? Source Code Map (128 Dart files)

Entry Points

File | Purpose

|---|---|

lib/main.dart | App bootstrap: Firebase, encryption, notifications, WorkManager, FCM, ads

lib/app.dart | MaterialApp.router with theme

lib/firebase_options.dart | Generated Firebase configuration

core/ ? Shared Infrastructure (42 files)

core/

??? app_theme.dart # Light/dark Material theme

??? constants/

? ??? app_constants.dart # App name, Firestore collection names

? ??? api_constants.dart # REST API endpoints

```

? ??? route_constants.dart      # All route paths
? ??? ads_constants.dart        # AdMob unit IDs
? ??? cloudinary_constants.dart  # Cloudinary cloud/preset
??? models/
? ??? job_model.dart            # Job data model (Firestore + REST)
? ??? chat_message.dart        # Chat message model
? ??? user_role.dart           # Enum: user, company, admin
??? network/
? ??? dio_client.dart          # HTTP client (Dio)
? ??? network_exceptions.dart   # API error mapping
??? providers/
? ??? core_providers.dart       # All Riverpod service providers
? ??? theme_provider.dart       # Dark/light mode state
??? router/
? ??? app_router.dart           # GoRouter routes + auth guards
? ??? router_refresh.dart       # Auth-aware router refresh
??? security/
? ??? aes_encryption_service.dart # AES-256 CTR encrypt/decrypt
? ??? secure_storage_service.dart # Key storage
??? services/
? ??? firebase_auth_service.dart # Firebase Auth wrapper
? ??? firestore_service.dart     # Firestore collection accessors
? ??? job_repository.dart        # Hybrid Firestore + REST jobs
? ??? chat_service.dart          # Chat CRUD + messaging
? ??? cloudinary_service.dart     # Image upload to CDN
? ??? fcm_service.dart           # Push notifications
? ??? local_notification_service.dart # Local alerts
? ??? ads_service.dart           # Google Mobile Ads
? ??? permission_service.dart    # Runtime permissions
? ??? performance_service.dart   # Firebase Performance traces
? ??? workmanager_service.dart   # Background tasks
? ??? isolate_service.dart       # Dart isolates
? ??? app_diagnostics_service.dart # Rubric health checks
? ??? toast_service.dart         # Toast notifications
??? utils/
? ??? app_logger.dart            # Structured logging
? ??? debug_log_store.dart       # In-memory log buffer
? ??? result.dart                # Success/Failure type
? ??? extensions.dart            # BuildContext helpers
??? widgets/
    ??? glassmorphic_app_bar.dart # Blur AppBar
    ??? rozgar_drawer.dart         # Side navigation drawer
    ??? rozgar_shell_body.dart     # AppBar safe area padding
    ??? sliding_job_card.dart      # Animated job card
    ??? banner_ad_widget.dart      # AdMob banner
    ??? skippable_ad_overlay.dart  # Full-screen ad
    ??? profile_avatar_picker.dart # Photo upload widget
    ??? media_permission_helper.dart # Permission rationale UI
    ??? async_error_view.dart      # Retry error state
    ??? shimmer_skeleton.dart      # Loading placeholder
    ??? animated_switcher_widget.dart # Tab transition animation

```

features/auth/ ? Authentication (12 files)

```

auth/
??? constants/auth_constants.dart # Sensitive field names
??? model/app_user.dart           # App user model
??? provider/
? ??? auth_provider.dart          # AuthNotifier (Riverpod)
? ??? auth_repository.dart        # Sign in/up/out + Firestore profile

```

```
??? screens/
?   ??? splash_screen.dart
?   ??? login_screen.dart      # Login + error banner
?   ??? register_screen.dart   # Register + role picker
??? utils/auth_navigation.dart # Role-based navigation helper
??? widgets/auth_text_field.dart # Styled text field
```

features/user/ ? Job Seeker (22 files)

```
user/
??? constants/user_constants.dart
??? model/
?   ??? job_application.dart    # Application + status enum
?   ??? user_profile.dart
?   ??? job_comment.dart
??? provider/
?   ??? user_jobs_provider.dart # Hybrid jobs feed
?   ??? application_provider.dart # Submit + status streams
?   ??? user_profile_provider.dart # Profile CRUD + encryption
?   ??? user_chat_provider.dart # Chat streams
?   ??? social_provider.dart    # Likes + comments
??? screens/
?   ??? user_home_screen.dart   # 4-tab shell
?   ??? user_jobs_screen.dart   # Job feed
?   ??? user_job_detail_screen.dart # Detail + apply + message
?   ??? user_applications_screen.dart # My applications
?   ??? user_chat_list_screen.dart # Conversations
?   ??? user_chat_screen.dart   # Chat thread
?   ??? user_profile_screen.dart # Profile editor
??? widgets/
    ??? application_status_tracker.dart
    ??? job_application_sheet.dart
    ??? comments_sheet.dart
    ??? like_button.dart
    ??? job_detail_header.dart
```

features/company/ ? Recruiter (14 files)

```
company/
??? constants/company_constants.dart
??? model/company_profile.dart
??? provider/
?   ??? company_jobs_provider.dart
?   ??? company_applications_provider.dart
?   ??? company_job_creator_provider.dart
?   ??? company_profile_provider.dart
??? screens/
?   ??? company_home_screen.dart # 4-tab shell
?   ??? company_jobs_screen.dart
?   ??? company_job_creator_screen.dart # 3-step wizard
?   ??? company_applications_screen.dart
?   ??? company_profile_screen.dart
?   ??? company_chat_screen.dart
??? widgets/company_stat_card.dart
```

features/admin/ ? Administrator (10 files)

```
admin/
??? constants/admin_constants.dart
??? model/admin_stats.dart
??? provider/admin_provider.dart
??? screens/
?   ??? admin_home_screen.dart # 5-tab shell
```

? ??? admin_analytics_screen.dart
? ??? admin_users_screen.dart
? ??? admin_jobs_screen.dart
? ??? admin_diagnostics_screen.dart # Rubric + logs
? ??? admin_profile_screen.dart
??? widgets/registration_chart.dart

Key Design Decisions

Decision | Reason

|---|---|

Feature-first folders | Easy to find code per role; scalable

Riverpod over Provider | Better async/stream support, compile-safe

GoRouter | Declarative routing with auth redirects

Result<T> type | Explicit error handling without exceptions everywhere

Hybrid job feed | Demonstrates both Firebase AND external REST API

AES encryption | Protects PII even if Firestore is compromised

Cloudinary | Professional CDN for images (not Firebase Storage)

PART C ? Viva Questions & Answers

Section 1: General Project Questions

Q1: What is your project about?

A: Rozgar is a multi-role job portal mobile app built with Flutter. It has three roles ? Job Seeker, Recruiter, and Admin. Job seekers browse and apply for jobs, recruiters post jobs and manage applicants, and admins monitor the system.

Q2: Why did you choose Flutter?

A: Flutter provides single codebase for Android and iOS, hot reload for fast development, rich widget library for professional UI, and excellent Firebase integration through official plugins.

Q3: What is the architecture of your app?

A: Feature-first clean architecture. lib/core/ contains shared services, widgets, and utilities. lib/features/ contains role-specific modules (auth, user, company, admin). Each feature has its own providers, screens, models, and widgets. State management uses Riverpod with repository pattern.

Q4: How many screens does your app have?

A: Approximately 20+ screens: 3 auth screens, 7 user screens, 6 company screens, 6 admin screens, plus shared chat screen.

Q5: What external services does your app use?

A: Firebase (Auth, Firestore, FCM, Performance), Cloudinary (image CDN), JSONPlaceholder (REST API for jobs), Google AdMob (advertisements).

Section 2: Firebase & Database

Q6: What Firebase services do you use?

A: Firebase Authentication (email/password), Cloud Firestore (NoSQL database), Firebase Cloud Messaging (push notifications), and Firebase Performance Monitoring (profiling).

Q7: Explain your Firestore database structure.

A: Six main collections: users (profiles with roles), jobs (listings with comments subcollection), applications (with status history subcollection), and chats (with messages subcollection). All collections require authentication per security rules.

Q8: What are Firestore security rules?

A: Rules in firestore.rules control who can read/write data. Users can only edit their own profile. All authenticated users can read jobs. Chat messages are restricted to conversation participants only. Rules are deployed to Firebase with firebase deploy --only firestore:rules.

Q9: How does authentication work?

A: User registers with email/password via Firebase Auth. A Firestore profile document is created with role (user/company/admin). AuthNotifier listens to auth state changes. GoRouter redirects to the correct home screen based on role. Login errors are mapped to user-friendly messages.

Q10: What happens when login fails?

A: Firebase returns error codes like invalid-credential, wrong-password, user-not-found. auth_repository.dart maps these to clear messages. login_screen.dart shows a red error banner, highlights the relevant field, and shows a toast.

Section 3: Security & Encryption

Q11: How do you encrypt sensitive data?

A: AES-256 CTR encryption via the encrypt package. A random 256-bit key is generated on first launch and stored in Flutter Secure Storage. Fields like phone and salary are encrypted before Firestore write and decrypted on read.

Q12: Why encrypt data if Firestore already has security rules?

A: Security rules protect access but data is still readable in Firebase Console by project owners. Encryption ensures even database administrators cannot read sensitive PII without the device-specific key.

Q13: Where is the encryption key stored?

A: In Flutter Secure Storage (lib/core/security/secure_storage_service.dart), which uses Android Keystore and iOS Keychain ? hardware-backed secure storage.

Q14: What fields are encrypted?

A: phone, salary, and identityDocument as defined in auth_constants.dart.

Section 4: REST API & Networking

Q15: Which external REST API do you use?

A: JSONPlaceholder (<https://jsonplaceholder.typicode.com/posts>) ? a free fake REST API. Dio HTTP client fetches posts and maps them to job models.

Q16: How does the hybrid job feed work?

A: JobRepository.fetchHybridJobs() fetches Firestore premium jobs and REST API jobs independently. If one source fails, the other still loads. Results are merged with premium jobs sorted first, then by date.

Q17: Why use both Firestore and REST API?

A: To demonstrate integration with both Firebase (real-time database) and external REST APIs (standard HTTP networking), as required by the semester rubric.

Section 5: UI & UX

Q18: What UI design patterns did you use?

A: Glassmorphic AppBar (blur effect), Material Design 3 theming, staggered list animations, shimmer loading skeletons, bottom navigation shells per role, modal bottom sheets for forms, and responsive error states with retry buttons.

Q19: How does the application status tracker work?

A: ApplicationStatusTracker widget shows a horizontal pipeline: Applied ? Under Review ? Interview Scheduled ? Offered. It listens to a Firestore stream on the application document. Status updates in real-time when the company changes it. Falls back to cached status if stream fails.

Q20: Explain the job posting wizard.

A: Company Post Job has 3 steps: (1) Job title and description, (2) Location, salary, and tags, (3) Banner image upload to Cloudinary. Step indicator shows progress. Validation runs per step. Final publish writes to Firestore jobs collection.

Section 6: Notifications & Background

Q21: How do notifications work?

A: Two systems: (1) FCM for push notifications from Firebase server, (2) Local notifications via flutter_local_notifications for app-generated alerts. On application submit, a local notification fires immediately. Foreground FCM messages are converted to local notifications.

Q22: What is WorkManager?

A: Android background task scheduler. Rozgar uses it for: (1) one-off decrypt task after application submit, (2) periodic 6-hour sync task. Initialized in main.dart, defined in workmanager_service.dart.

Q23: What are Dart Isolates?

A: Separate memory threads for CPU-intensive work. IsolateService offloads JSON parsing and AES decryption so the UI thread stays smooth. Demonstrates threading without blocking the main isolate.

Section 7: Permissions & Ads

Q24: How do you handle permissions?

A: Three-step flow: (1) Rationale dialog explaining why permission is needed, (2) System Allow/Deny dialog, (3) Open Settings fallback if permanently denied. Implemented in MediaPermissionHelper and used for camera, gallery, and notifications.

Q25: How are ads integrated?

A: Google Mobile Ads SDK. Banner ad on all role home screens above bottom navigation. Skippable interstitial overlay on user home after 2.5 minutes with 5-second countdown and X close button. Test ad unit IDs used for development.

Section 8: Logging, Profiling & Debugging

Q26: How do you implement logging?

A: AppLogger with levels (debug, info, warn, error, severe, auth, network). Logs stored in DebugLogStore (100-entry ring buffer). Admin can view last 20 logs in System Diagnostics screen. Console output in debug mode via logger package.

Q27: What is Firebase Performance profiling?

A: Custom traces measure operation duration. Traces: auth_sign_in, fetch_hybrid_jobs, post_premium_job, clouinary_upload_*. Viewable in Firebase Console ? Performance tab.

Q28: How do you debug the app?

A: (1) flutter run console logs, (2) Admin System Diagnostics screen with live logs, (3) Firebase Console for Firestore data and performance, (4) AppLogger throughout codebase.

Section 9: Chat & Messaging

Q29: How does the chat system work?

A: ChatService manages Firestore chats and messages collections. Chat ID is generated by sorting two user IDs. ensureChat() creates chat document before messaging. Messages stream in real-time. Security rules restrict access to participants only.

Q30: Why was chat showing permission denied?

A: Messages were being read before the chat document existed, and security rules require participant membership. Fixed by creating chat document before messages and calling ensureChat() on chat screen open. Rules deployed to Firebase.

Section 10: Technical Deep-Dive

Q31: What is Riverpod and why use it?

A: Riverpod is a reactive state management library for Flutter. It provides Provider, StateNotifierProvider, StreamProvider, and FutureProvider. Used for dependency injection, auth state, Firestore streams, and feature state. Better than Provider because it is compile-safe and supports async natively.

Q32: What is GoRouter?

A: Declarative routing package. Routes defined in app_router.dart with path parameters (:jobId, :chatId). Auth redirect guard sends unauthenticated users to login and authenticated users to role home.

Q33: What is the Result type pattern?

A: Sealed class with Success<T> and Failure<T> variants. Repository methods return Result instead of throwing exceptions. UI calls .when(success: ..., failure: ...) for type-safe error handling.

Q34: How does Cloudinary upload work?

A: Unsigned upload preset sends image bytes via HTTP POST to Cloudinary API. Returns secure CDN URL stored in Firestore. Used for profile photos, job banners, and resumes. Performance trace wraps each upload.

Q35: What would you improve in future?

A: (1) Email verification, (2) Password reset flow, (3) Video interviews, (4) AI job matching, (5) Payment integration for premium listings, (6) Offline caching with Hive/SQLite, (7) Unit and integration tests.

Section 11: Demo Script (5-Minute Presentation)

Time | Action | Highlight

|---|---|---|

0:00 | Open app ? Splash ? Login as User | Auth, GUI

0:30 | Browse jobs ? tap job ? like/comment | REST+Firestore, GUI

1:00 | Apply to job ? notification | Notifications, Firebase DB

1:30 | Check Applications ? status tracker | Real-time streams

2:00 | Message recruiter | Chat, Firestore rules

2:30 | Switch to Company ? Post Job | GUI, Cloudinary, Permissions

3:00 | Review applicants ? update status | CRUD, status tracking

3:30 | Switch to Admin ? System tab | All rubric items verified

4:00 | Show Firebase Console (data, performance) | Firebase, Profiling

4:30 | Show encrypted field in Firestore | Security

5:00 | Q&A | Viva

Section 12: Common Viva Traps ? Be Ready

Trap Question | Correct Answer

|---|---|

"Show me encryption working" | Open Firebase Console ? users ? phone field is ciphertext, not readable number

"Where is REST API code?" | job_repository.dart line calling Dio ? JSONPlaceholder

"Show background task" | Apply to job ? check logs for WorkManager task scheduled

"Show profiling" | Firebase Console ? Performance ? custom traces

"What if internet is off?" | Jobs show retry button; Firestore cache may show stale data

"Is this production-ready?" | Uses test ad IDs and unsigned Cloudinary preset; would need production keys for release

Document: Rozgar System Overview & Viva Guide v1.0

Total Dart Files: 128 | Roles: 3 | Firestore Collections: 4 (+ 3 subcollections)

Generated for: Semester 6 ? Mobile Application Development